

Distribuciones tipo Fase

Alejandro García de León

2026-02-01

Modelación de reclamaciones

```
library(readr)
reclamaciones_simuladas <- read_csv("C:/Users/Alejandro/Downloads/WPy64-38123/notebooks/reclamaciones_s
  col_types = cols(fecha_ocurrencia = col_date(format = "%Y-%m-%d")))
```

```
head(reclamaciones_simuladas)
```

```
## # A tibble: 6 x 4
##   fecha_ocurrencia ramo           cliente_id monto_reclamacion
##   <date>           <chr>         <chr>         <dbl>
## 1 2018-01-01       Auto           C003874       25690.
## 2 2018-01-01       Auto           C000756       24195.
## 3 2018-01-01       Vida           C004535      150428.
## 4 2018-01-01       Gastos Médicos C002407       41081.
## 5 2018-01-01       Daños          C005122       74334.
## 6 2018-01-01       Daños          C000685      185093.
```

Distribuciones tipo fase

Las distribuciones tipo fase es un conjunto denso para las distribuciones positivas. Se sabe que existe una sucesión de distribuciones tipo fase que converge a la distribución objetivo. Primero es necesario contar con una distribución para las reclamaciones. Usaremos la función de distribución empírica como una aproximación:

```
library(dplyr)
```

```
##
## Adjuntando el paquete: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

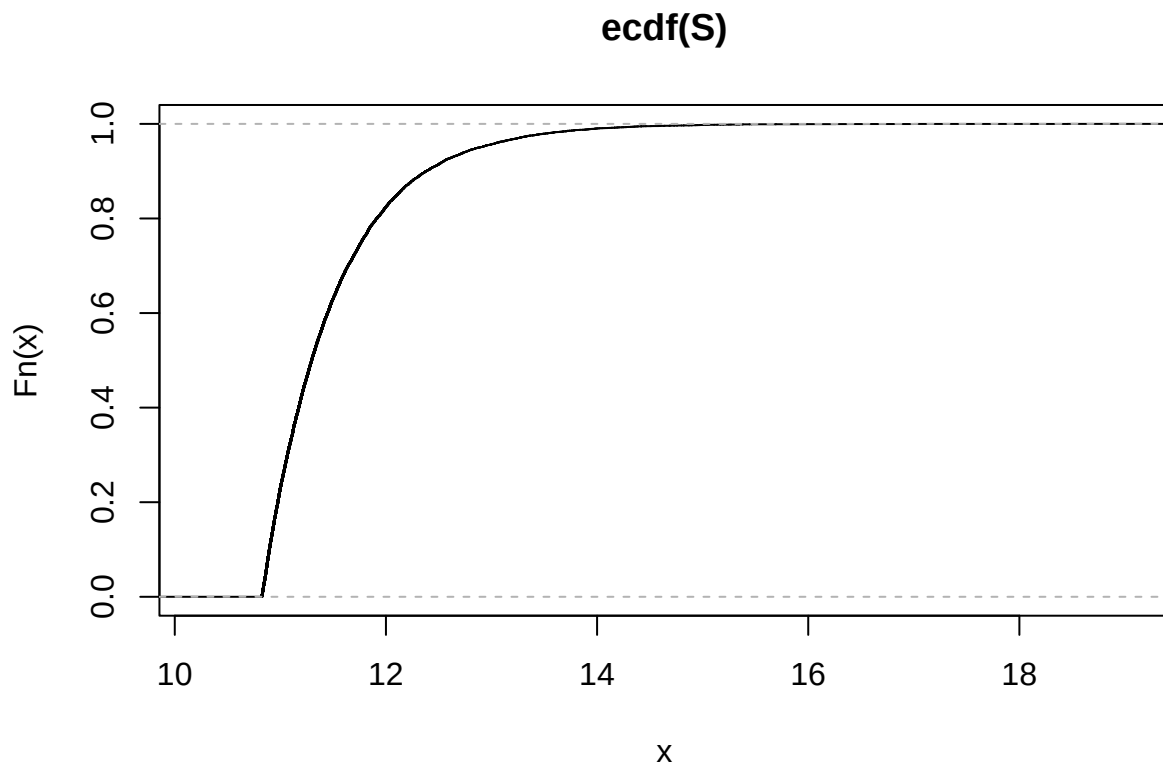
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
unique(reclamaciones_simuladas$ramo)
```

```
## [1] "Auto"          "Vida"           "Gastos Médicos" "Daños"
```

```
S = reclamaciones_simuladas[>%filter(reclamaciones_simuladas$ramo == "Daños")]
S = S$monto_reclamacion
```

```
S=log(S)
plot(ecdf(S))
```



```
range(S)
```

```
## [1] 10.81980 18.45526
```

Ahora bien la sucesión aproximante toma la siguiente forma

```
n = 100
tamaño = n^2
x = c(1:n^2)
x = x/n
x = ecdf(S)(x)
p = x
p = sort(p)
```

```

p = diff(p)
a = min(S)
Erlang = function(y){
  if(y<10){
    return(0)
  }else{
    s = c(1:length(p))
    s = sapply(s, function(z) pgamma(y,shape = z ,rate = n))
    s = sum(p*s)
    s = s+ecdf(S)(a)+1-ecdf(S)(length(p))
    return(s)}
}

```

```

set.seed(1234)
M = S
#Kernel normal
cociente = function(x,h,M){
  return((x-M)/h)
}
kernel1= function(x,h,M){
  sumando = sum(dnorm(cociente(x,h,M),mean = 0, sd=1))
  return(sumando/(length(M)*h))
}
#La constante de escalamiento
h = bw.nrd0(M)

```

```

#Límites del eje x
a=min(M)
b=max(M)
dominio = seq(from = a, to = b, length.out= 1000)
#Densidad suavizada objetivo a modelar
#Aplicar el kernel al dominio
ya = sapply(dominio, FUN = kernel1,h=h,M=M)
set.seed(1234)
distr = function(b){
  d = runif(1000,a,b)
  int = mean(sapply(d, function(x) kernel1(x, h = h, M=M)))/dunif(d,a,b))
  return(int)
}

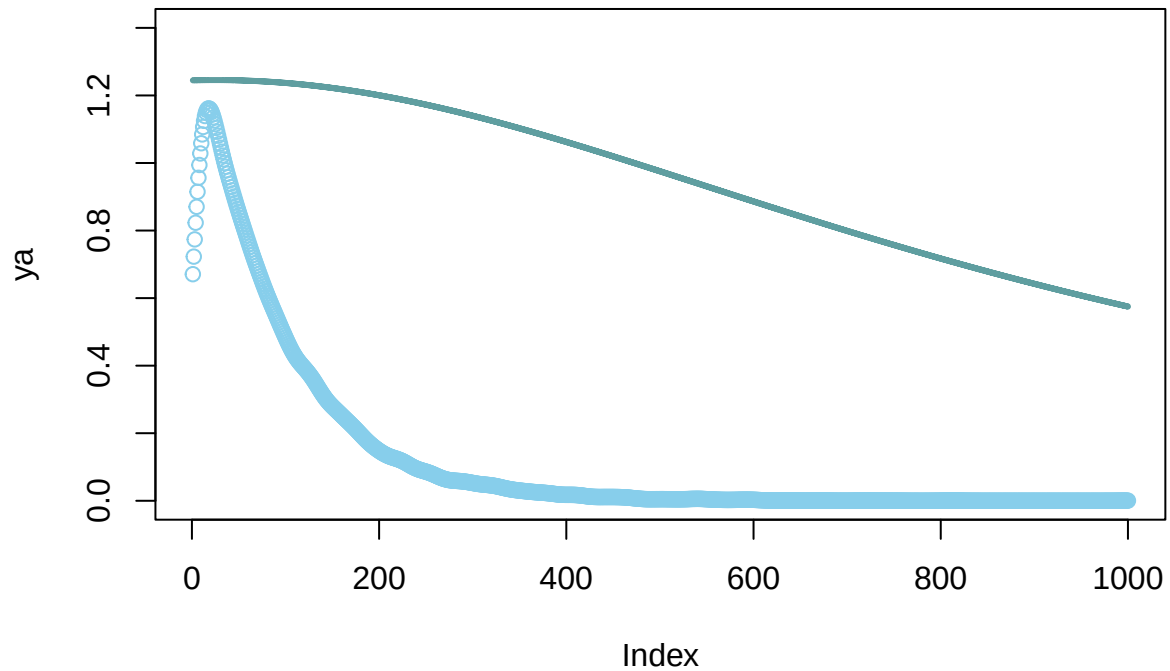
```

```

#Límites del eje x
a=min(M)
b=max(M)
dominio = seq(from = a, to = b, length.out= 1000)
#Densidad suavizada objetivo a modelar
#Aplicar el kernel al dominio
ya = sapply(dominio, FUN = kernel1,h=h,M=M)
#Densidad que acota por arriba a la densidad objetivo
#Aplicar densidad Cauchy al dominio
ye = sapply(dominio, FUN = dcauchy,location = 11, scale =6.9)
ye = 27*ye

```

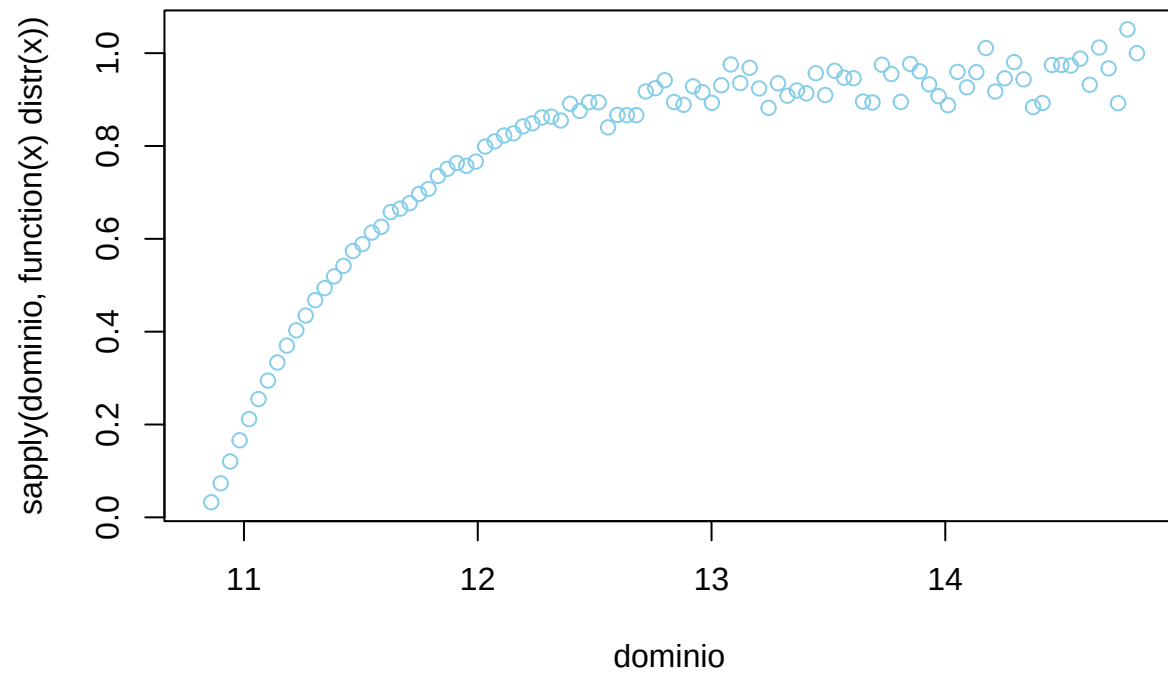
```
#curve(dlnorm(x,m,m1))
plot(ya, ylim=c(0,1.4), col="skyblue")
lines(ye,lwd = 3,type = "s", col = "cadetblue")
```



Para probar la anterior aproximación se usará la función recién definida

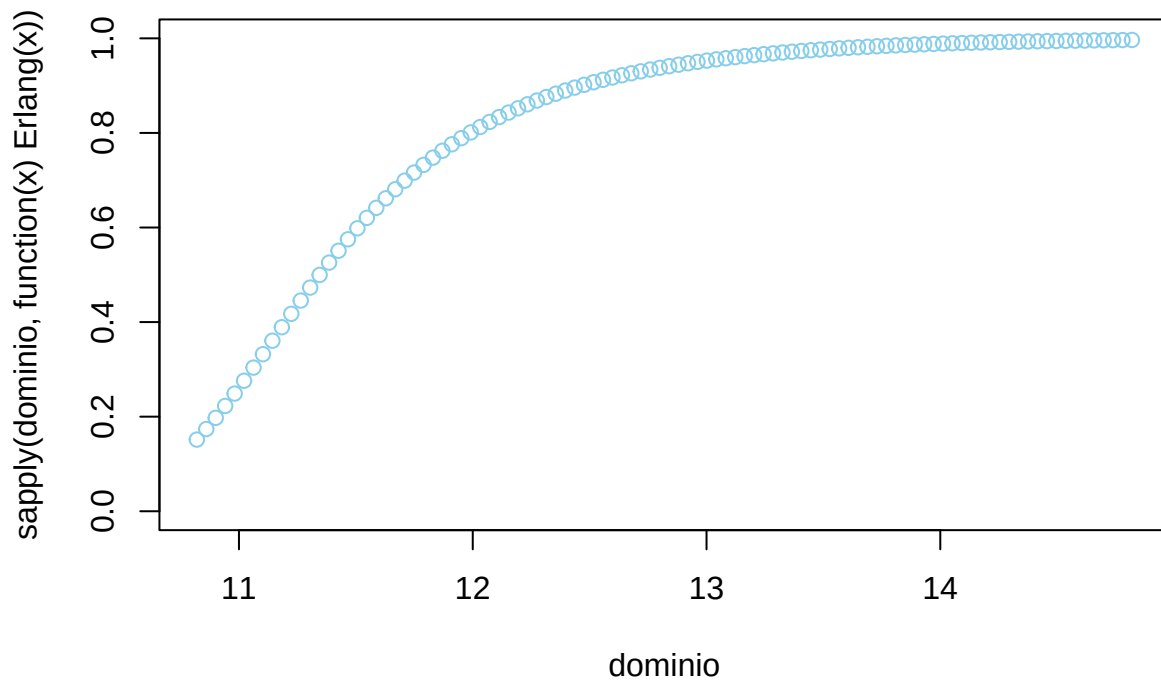
```
#max(ecdf(S)(dominio)-sapply(dominio, function(x) Erlang(x)))
a=min(M)
b=min(M)+4
dominio = seq(from = a, to = b, length.out= 100)
plot(dominio,sapply(dominio, function(x) distr(x)),col="skyblue")
```

```
## Warning in dunif(d, a, b): Se han producido NaNs
```



```
#points(ecdf(S)(dominio),col = "blue")
```

```
a=min(M)
b=min(M)+4
dominio = seq(from = a, to = b, length.out= 100)
plot(dominio,sapply(dominio, function(x) Erlang(x)), ylim = c(0,1),col="skyblue")
```



```
set.seed(123)

F0 <- function(x) Erlang(x)
r0 <- function(n) sample(S,n)

F1 <- function(x) sapply(x, function(b) Erlang(b))

n <- 50
B <- 100

# KS observado
x_obs <- r0(n)
Fn <- ecdf(x_obs)
grid <- sort(x_obs)

D_obs <- max(abs(Fn(grid) - F1(grid)))

# Bootstrap KS
D_boot <- numeric(B)

for (b in 1:B) {
  x <- r0(n)
  Fn_b <- ecdf(x)
  D_boot[b] <- max(abs(Fn_b(grid) - F1(grid)))
}
```

```
p_value <- mean(D_boot >= D_obs)
p_value
```

```
## [1] 0.22
```

Un poco de teoría

Probabilidad de ruina

El cálculo de reservas técnicas puede ser realizado de varias formas, una es a través de un proceso de riesgo definido como un proceso de Poisson:

$$R_t = u + \beta \cdot t - S(t)$$

Aquí $S(t) = \sum_{i=1}^{N_t} Z_i$ es el modelo de Cramer Lundberg donde $Z_i = \log(X_i)$ es la transformada bajo Box Cox con la cual hemos encontrado una excelente bondad de ajuste.

Fórmula Pollackzek Kinchin

Mediante la fórmula $\psi(u) = (1 - p) \sum_{i=1}^{\infty} p^i \cdot \bar{H}^*(u)$ para un capital inicial u no negativo donde p (también se le llama ρ) es la condición de ganancia neta $\frac{\lambda\mu}{\beta} < 1$ y $H(u) = \frac{\int_{0,u} \bar{F}(y) dy}{\mu}$ es la función de riesgo integrado.

Esta fórmula puede ser complicada de aproximar por la convolución pero cuando las distribuciones son tipo fase $PH(B, \alpha)$ la fórmula de Pollackzek Kinchin se vuelve en una forma cerrada aunque matricial.

$$\psi(u) = \rho \cdot \alpha^s \exp(u(B + \rho b^t \alpha^s)) e^t,$$

aquí, $e = (1, 1, \dots, 1)$. Para calcular esto, debe calcularse $\exp(u(B + \rho b^t \alpha^s))$.

Esta debería ser igual a

$$\exp(uB) \exp(\rho b^t \alpha^s),$$

Sin embargo esto solo ocurre cuando ambos conmutan. es decir pertenecen al centralizador y son simultáneamente diagonalizables. Por ende debemos usar una aproximación.

Se eligió **strang splitting** con lo que pueden calcularse por separado $\exp(uB)$ y $\exp(\rho b^t \alpha^s)$.

$$\exp(A + B) \sim \exp(A/2) * \exp(B) * \exp(A/2),$$

el cual en el caso de una suma decrece en orden $O(\frac{1}{n^2}) = O(1)$ para $n=1$.

Con esta técnica el ajuste se hará en dos partes.

Calculo de $\exp(uB)$

```

rm(exp)
u = 100000000
media = mean(exp(S))
recargo = (1-(.18+.42+.3))
media = (1/recargo)*media
beta = media
##Transformación de la prima y el capital
u_prime = log(u)
beta_prime = u/beta
##Media de las reclamaciones logarítmicas
mu=mean(S)
#Media del proceso de Poisson a t=1
lambda=45
##Condición de ganancia neta
C_ganacia_neta = lambda*mu
C_ganacia_neta = C_ganacia_neta/beta
###Debe ser menor a 1
C_ganacia_neta<1

```

```
## [1] TRUE
```

Como B esta en forma canónica de Jordan transpuesta,

$$B = \begin{pmatrix} \lambda & 0 & 0 & \cdots & 0 \\ 0 & \ddots & & & \\ 0 & & \ddots & & \\ \vdots & & & \lambda & 0 \\ 0 & & & 0 & 0 & 1 \\ & & & & 0 & 0 & \ddots \\ & & & & \ddots & \ddots & 1 \\ & & & & & 0 & 0 \end{pmatrix}$$

entonces claramente $B = -\lambda \cdot Id + \lambda \cdot U$ donde U es una matriz triangular superior y nilpotente.

A partir del bloque mas grande todos los bloques se han hecho cero, es decir si

$$\exp(\lambda \cdot U) = \sum_{i=0}^{\infty} \frac{\lambda^i U^i}{i!},$$

entonces esto es equivalente a

$$\exp(\lambda \cdot U) = \sum_{i=0}^{n-1} \frac{\lambda^i U^i}{i!}$$

y esto igual a

$$\sum_{i=0}^{n-1} \frac{\lambda^i}{i!},$$

es decir el último valor distinto de 0.


```

#Para ahorrar memoria solo se trabajarán
#con pesos donde la mezcla sea mayor a 0
m12 = which(p!=0)
#vector con los tamaños de los bloques de Jordan
n_vec <-m12
#Se procederá a calcular alfa_s
k=m12[1]
cs = rep(0, times = k)
  cs[1] = 1
  cs = p[k]*cs
  alfa = cs
for(k in 2:length(n_vec)){
  cs = rep(0, times = m12[k])
  cs[1] = 1
  cs = p[m12[k]]*cs
  alfa = c(alfa,cs)
}
#Se valida que sea distribución de probabilidad
sum(alfa)

```

```
## [1] 1
```

```

#Este código es alternativo y permite calcular la exponencial por posición
#si no fuera muy grande la matriz
#exponencial = matrix(nrow =nrow(J_dense), ncol =ncol(J_dense))
#for(j in 1:ncol(J_dense)){
#for(m in 1:nrow(J_dense)){
#  if(J_dense[m,j]!=0){
#    x1 =lgamma(abs(j-m) + 1)
#    x1 =exp(x1)
#exponencial[m,j] = ((u_prime*J_dense[m,j])^(j-m))/x1
#  }else{
#exponencial[m,j] = 0
#}
#}
#}

```

Por ser una matriz triangular superior para entrada $i, i+k$, y solo contar con un valor 1, queda de la forma $\frac{\lambda^k}{k!}$ para la entrada $(i, i+k)$.

```

library(Matrix)
#Vector con los eigenvalores de cada bloque de Jordan
lambda_vec <- rep(100, times = length(n_vec))
#Se crea un vector de bloques de Jordan aplicando
#La función implementada para ello jordan_block
blocks <- mapply(
  jordan_block,
  n = n_vec,
  lambda = lambda_vec,
  SIMPLIFY = FALSE
)
#Se genera una matriz diagonal por bloques
J <- bdiag(blocks)

```

```
#Si se desea y el tamaño lo permitiera podría hacerse densa para su visualización
#J_dense <- J#as.matrix(J)
```

Ahora bien para calcular $\exp(-\lambda \cdot Id)$ esta se puede reducir a la matriz cuya diagonal contiene a $e^{-\lambda}$ que es $e^{-\lambda} \cdot Id$.

Ahora por tanto la forma canónica esta dada por

$$\exp(uB) = e^{-\lambda} \sum_{i=0}^{n-1} \frac{\lambda^i}{i!}.$$

Como nota los parámetros se han cambiado en escala dado el uso de Z_i en vez de X_i para destransformar al final

```
#Usando la matriz de Jordan, se calculará
#la exponencial
library(Matrix)
J <- bdiag(blocks) # dgCMatrix triangular superior
#La exponencial se llamará E
E <- J
i <- E@i + 1 # filas (1-based)
j <- rep(seq_len(ncol(E)), diff(E@p)) # columns

k <- j - i # j - m
valid <- k >= 0
E@x <- numeric(length(E@x))
E@x[valid] <- (exp(-100)*(u_prime * J@x[valid])^k[valid]) /
  exp(lgamma(k[valid] + 1))
#Vector sparse
v_sparse <- as(alfa, "sparseVector")
#Matriz de una columna con sparse
v_sparse = Matrix(alfa, nrow = length(alfa), ncol = 1, sparse = TRUE)
sum(v_sparse)
```

```
## [1] 1
```

```
#Se almacena el resultado
JER = E
#Se rescata la matriz de Jordan como matriz triangular superior
JER_tri <- as(J, "dtCMatrix")
class(JER_tri)
```

```
## [1] "dtCMatrix"
## attr("package")
## [1] "Matrix"
```

```
slot(JER_tri, "uplo")
```

```
## [1] "U"
```

```

#alfa_s = (-1/mu)*(forwardsolve(JER_tri,v_sparse))
#JER_tri@uplo
#Se creará alfa_s a partir de los insumos anteriores
alfa_s <- numeric(sum(n_vec))
#Indices acumulados
block_sizes = n_vec
ends <- cumsum(block_sizes)
starts <- c(1, head(ends, -1) + 1)
for (i in seq_along(n_vec)) {

  idx <- starts[i]:ends[i]

  alfa_s[idx] <- (-1/mu) * backsolve(
    JER_tri[idx, idx],
    as.numeric(v_sparse[idx])
  )
}
alfa_s =t(t(alfa_s))
#Estas son solo validaciones internas
#alfa_s <- (-1/mu) * solve(t(JER_tri), v_sparse)
#min(alfa_s)
#alfa_s@x
#alfa_s = v_sparse%*%alfa_s
#nrow(v_sparse)
#ncol(alfa_s)
head(alfa_s)

```

```

##           [,1]
## [1,] 4.162436e-07
## [2,] 0.000000e+00
## [3,] 0.000000e+00
## [4,] 0.000000e+00
## [5,] 0.000000e+00
## [6,] 0.000000e+00

```

Segunda exponencial: $\exp(\rho b^t \alpha^s)$

Para esta parte se debe dividir entre 2 $\exp(\frac{\rho b^t \alpha^s}{2})$, pues no conmutan ambas matrices de manera evidente. Por ende los cálculos serán aproximados.

La k-esima potencia de esta matriz con cero excepto en el último renglón será:

$$R_{n,n}^k(\rho b^t \alpha^s / 2) = a_n^k \cdot \begin{pmatrix} 0 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ a_1 & a_2 & \cdots & a_n \end{pmatrix}$$

Si la última entrada es 0

$$e^{(\rho b^t \alpha^s / 2)} = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ a_1 & a_2 & \cdots & 1 \end{pmatrix}$$

```

n123= nrow(E)
v <- Matrix(0, n123, 1, sparse = TRUE)
#v[n123, 1] <- 100
v[m12,1] = 100

#####segunda exponencial
exp2 = u_prime*((C_ganacia_neta*v)/2)
exp2 = exp2%*%Matrix(as.numeric(alfa_s), byrow = FALSE, ncol = length(alfa_s), nrow = 1, sparse = TRUE)
#summary(exp2)
#summary(Matrix(as.numeric(alfa_s), byrow = FALSE, ncol = length(alfa_s), nrow = 1, sparse = TRUE))
#exp2 = exp(exp2[nrow(exp2),ncol(exp2)])
ser = Diagonal(x = NULL, n = nrow(exp2))
exp2 = ser+exp2

```

En caso contrario

$$\begin{array}{cccc}
 1 & 0 & \cdots & 0 \\
 0 & 1 & \cdots & 0 \\
 \vdots & \vdots & \ddots & \vdots \\
 \frac{e^{a_n} - 1}{a_n} a_1 & \frac{e^{a_n} - 1}{a_n} a_2 & \cdots & e^{a_n}
 \end{array}$$

Unir ambos resultados

Finalmente para calcular la exponencial definitiva

```

#Exponencial definitiva
#Esta es la aproximación
ExpDef = E
ExpDef = exp2%*%E
ExpDef = ExpDef%*%exp2
#Se combina con la fórmula de pollackzek kinchin
XS =t(alfa_s*C_ganacia_neta)
XS = XS%*%ExpDef
####Estas son solo validaciones
#unique(as.numeric(XS))
#ExpDef%*%t(t(rep(1, times =ncol(ExpDef))))
#Esta es la probabilidad de ruina
XS = XS%*%t(t(rep(1, times =ncol(XS))))
XS

```

```

## 1 x 1 Matrix of class "dgeMatrix"
##           [,1]
## [1,] 1.932547e-47

```

Recuperación del modelo original

Si bien se cálculo la probabilidad de ruina para las transformadas usando la aproximación:

$$\log(1+t) \sim 1+t$$

Para t pequeño, recordamos que de la fórmula de Pollackzek Kinchin

$$\psi(u) = (1-p) \sum_{i=1}^{\infty} p^i \cdot \tilde{H}^*(u)$$

La función de riesgo integrado solo depende de la convolución de F la función de distribución original quien es una transformación del logaritmo $Z_i = \log(X_i)$ y por ende $F_X(x) = F_Z(\log(x))$. De esta forma la probabilidad de ruina que es el infimo del tiempo en que ocurre ($R'_t < 0$) o bien ($u' + \beta' \cdot t < S'_t$) donde S'_t es el modelo con reclamaciones transformadas, coincide con el evento de manera aproximada a ($\beta \cdot t + u \sim \log(u' + \beta' \cdot t) < S_t$) que es la probabilidad de ruina original por construcción.